SUMARY: MODULES

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand what modules are
- Understand how to create and use a module
- Understand how to rename a module
- Understand the built-in modules
- Understand what file handling is

KEY POINTS AND EXAMPLES:

Understand what modules are

A module in Python is simply a Python file with the .py extension that contains Python code (variables, functions, etc.).

The main purpose of a module is to organize and reuse code across multiple programs or parts of a program. By modularizing code, you can break down complex programs into smaller, more manageable parts that can be shared and maintained more easily.

Imagine you have a toolbox filled with different tools. Each tool is designed to perform a specific task, like a hammer for driving nails, a screwdriver for tightening screws, or a wrench for turning nuts and bolts.

A Python module is like a specialized tool in your toolbox. It contains a set of related functions and variables that are designed to perform specific tasks or solve a particular problem.

For example, the math module in Python is like a tool for performing mathematical operations. It contains functions for calculating square roots, trigonometric functions, logarithms, and more. Instead of writing all these complex mathematical formulas yourself, you can simply import the math module and use its pre-built functions, just like grabbing a calculator tool from your toolbox.

Another example is the os module, which is like a tool for interacting with the operating system. It provides functions for creating directories, deleting files, checking file paths, and performing other system-related tasks.

Just like how you can have multiple tools in your toolbox, you can have multiple modules in your Python program. And just like how you can combine different tools to accomplish a larger task, you can import and use multiple modules together in your Python code.

When you need a specific functionality, instead of writing all the code from scratch, you can simply import the relevant module (tool) into your program and use its functions or variables to get the job done quickly and efficiently.

Additionally, just as you can create your own custom tools and add them to your toolbox, you can create your own modules in Python and reuse them across different projects or share them with others.

Understand how to create and use a module

1. **Create a new Python file:** Start by creating a new Python file with a .py extension, let's say my_module.py. You can create this file in the same directory where your Jupyter Notebook is located, or in a separate directory.

2. **Define your module code:** In the my_module.py file, define your functions, variables, or any other code you want to include in the module, just like you would in a regular Python script. For example,

```
def greeting(name):
  print("Hello, " + name)
```

3. **Create a new Jupyter Notebook**: Open Jupyter Notebook and create a new notebook.

4. **Import the module in your notebook:** In one of the code cells in your Jupyter Notebook, import your module using the import statement.

```
import my_module
```

5. **Use the module components:** Once you've imported the module, you can use its components (functions, variables, etc.) in your Jupyter Notebook code cells.

```
# Use a function from the module
my_module.greeting("Alice")
```

6. **Run the code cells:** Execute the code cells in your Jupyter Notebook to see the output or results of using your module.

The output will be:

```
Hello, Alice
```

You can also create modules directly within a Jupyter Notebook by defining your functions, classes, and variables in code cells, and then importing them in other code cells within the same notebook. However, it's generally recommended to create separate .py files for your modules, as it promotes better code organization and reusability.

Save this code in the file math_utils.py

```
# math_utils.py

"""
This module provides utility functions for common
mathematical operations.
"""

def square(x):
    """
    Returns the square of a given number.
    """
    return x ** 2

def cube(x):
    """
    Returns the cube of a given number.
    """
    return x ** 3

def factorial(n):
    """
    Returns the factorial of a given positive integer.
    """
    if n < 0:
        raise ValueError("Factorial is not defined for negative
numbers.")
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

In this math_utils.py module, we define three functions:

- square(x): Returns the square of a given number x.
- cube(x): Returns the cube of a given number x.
- factorial(n): Returns the factorial of a given positive integer n.

You can import and use these functions in your Jupyter Notebook

```
import math_utils

result1 = math_utils.square(5)
print(f"Square of 5: {result1}")  # Output: Square of 5: 25

result2 = math_utils.cube(3)
print(f"Cube of 3: {result2}")  # Output: Cube of 3: 27

result3 = math_utils.factorial(5)
print(f"Factorial of 5: {result3}")  # Output: Factorial of 5: 120
```

Save this code in the file string_utils.py

```
# string_utils.py

"""
This module provides utility functions for working with strings.
"""

def reverse_string(s):
    """
    Returns the reverse of a given string.
    """
    return s[::-1]

def count_vowels(s):
    """
    Returns the count of vowels (a, e, i, o, u) in a given string.
    """
    vowels = "aeiou"
    count = 0
    for char in s.lower():
        if char in vowels:
            count += 1
    return count
```

In this string_utils.py module, we define two functions:

- reverse_string(s): Returns the reverse of a given string s.
- count_vowels(s): Returns the count of vowels (a, e, i, o, u) in a given string s.

You can import and use these functions in your Jupyter Notebook

```
import string_utils

result1 = string_utils.reverse_string("Hello, World!")
print(f"Reversed string: {result1}")  # Output: Reversed string:
!dlroW ,olleH

result2 = string_utils.count_vowels("Python")
print(f"Number of vowels in 'Python': {result2}")  # Output:
Number of vowels in 'Python': 1
```

Note: You can name the module file whatever you like, but it must have the file extension .py

To import a specific function from a module in Python:

Syntax:

```
from module import function
```

from module: This part of the syntax specifies the module from which you want to import something.

import function: This part specifies the particular function you want to import from the module. By importing only the specific function, you can use it directly in your code without needing to prefix it with the module name.

Assume you have a module named math_operations.py

```
# math_operations.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Division by zero!"
```

To use the add function from this module in your script, you can import it specifically:

```
# main.py
from math_operations import add

result = add(5, 3)
print(result)  # Output: 8
```

In this example:

- from math_operations specifies the module from which to import.
- import add specifies the particular function to import.
- After importing, you can use add directly without prefixing it with math_operations.

You can also import multiple specific functions by separating them with commas:

```
from math_operations import add, subtract

result1 = add(5, 3)
result2 = subtract(5, 3)

print(result1)  # Output: 8
print(result2)  # Output: 2
```

To import all functions from a module in Python

Syntax:

```
from module import *
```

Assume you have a module named math_utils.py

```
# math_utils.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero.")
```

```
    return a / b
```

If you want to import all functions from math_utils:

```
from math_utils import *

print(add(2, 3))       # Output: 5
print(subtract(5, 2))  # Output: 3
print(multiply(3, 4))  # Output: 12
print(divide(10, 2))   # Output: 5.0
```

Understand how to rename a module

By default, when you import a module in Python, you use the name of the Python file (without the .py extension) as the module name. However, you can rename a module or give it an alias when importing it using the import ... as ... syntax.

Suppose we have a module named calculator.py

```
# calculator.py

def add(a, b):
    return a + b
def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero!")
    return a / b
```

Now, let's import this module in another Python file and rename it using the import ... as ... syntax:

```
# Import the calculator module and rename it to 'calc'
import calculator as calc

# Use the functions from the renamed module
```

```
result1 = calc.add(2, 3)
print(f"2 + 3 = {result1}")  # Output: 2 + 3 = 5

result2 = calc.subtract(5, 2)
print(f"5 - 2 = {result2}")  # Output: 5 - 2 = 3

result3 = calc.multiply(4, 6)
print(f"4 * 6 = {result3}")  # Output: 4 * 6 = 24

result4 = calc.divide(10, 2)
print(f"10 / 2 = {result4}")  # Output: 10 / 2 = 5.0
```

Note: It's important to note that when you rename a module using import ... as ..., you are creating an alias for the module, not changing the original module name. The module's actual name (in this case, calculator.py) remains the same.

By renaming the module when importing, you can:

- Give a more meaningful or concise name to the module for better readability in your code.
- Avoid naming conflicts if you have multiple modules with the same name.
- Make your code more organized and easier to understand, especially when working with third-party modules or packages with long or cryptic names.

you can use an alias when importing a specific function from a module.

```
# math_operations.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Division by zero!"
```

To import and use the add function with an alias:

```
from math_operations import add as addition

result = addition(5, 3)
print(result)  # Output: 8
```

You can also use this technique to give shorter names to modules with long or verbose names, making your code more concise and readable.

Suppose we have a module named very_long_module_name.py

```
# very_long_module_name.py

def greet(name):
    print(f"Hello, {name}!")
```

Now, let's import this module in another Python file and give it a shorter alias using the import ... as ... syntax:

```
# Import the module with a long name and give it a shorter alias
import very_long_module_name as vlmn

# Use the functions from the renamed module
vlmn.greet("Alice")  # Output: Hello, Alice!
```

Suppose we have two modules named utils.py from different directories:

```
# dir1/utils.py

def greet(name):
    print(f"Hello, {name}!")

def get_sum(a, b):
    return a + b
```

```
# dir2/utils.py

def reverse_string(s):
    return s[::-1]

def is_palindrome(s):
    return s == s[::-1]
```

If we try to import both of these modules using their original names in the same Python file, we'll encounter a naming conflict:

```
import dir1.utils
import dir2.utils  # Error: Attempted relative import with no
known parent package

# This will cause a naming conflict
package1.utils.greet("Alice")
package2.utils.reverse_string("Hello")
```

To avoid this naming conflict, we can rename one or both of the modules using the import ... as ... syntax:

```
# Importing utils.py from dir1 with an alias
import dir1.utils as utils1

# Importing utils.py from dir2 with an alias
import dir2.utils as utils2

# Using the functions from dir1/utils.py
utils1.greet("Alice")  # Output: Hello, Alice!
print(utils1.get_sum(3, 4))  # Output: 7

# Using the functions from dir2/utils.py
print(utils2.reverse_string("hello"))  # Output: olleh
print(utils2.is_palindrome("racecar"))  # Output: True
```

Why Use dir1 and dir2?

- Organizational Purpose: Directories help organize code by grouping related functionalities together. This is especially useful in large projects.
- Avoiding Conflicts: By placing each utils.py in a separate directory, Python can distinguish between them, allowing you to import both modules without conflict.
- Importing with Aliases: Aliases (utils1 and utils2) provide a convenient way to refer to these modules in your code, making it clear which module is being used.

Understand the built-in modules

There are several built-in modules in Python, which you can import whenever you like.

1. **platform module:** The platform module in Python provides functions and variables that give information about the system on which the Python interpreter is running.

a. Getting System Information:

```
import platform

print(platform.system()) # Output: Windows, Linux, Darwin (macOS),
Java (Jython), etc.
print(platform.release()) # Output: OS release version
print(platform.version()) # Output: Detailed OS version
information
print(platform.machine()) # Output: Architecture (e.g., 'x86',
'x86_64', 'AMD64', 'PowerPC')
print(platform.processor()) # Output: Processor name
```

b. Determining Python Version and Implementation:

```
import platform

print(platform.python_version()) # Example output: '3.9.1'
print(platform.python_compiler()) # Example output: 'GCC 8.4.0'
print(platform.python_implementation()) # Example output:
'CPython'
```

c. Checking for Specific Features or Characteristics:

```
print(platform.architecture()) # Output: Tuple with architecture
and bit size (e.g., ('64bit', 'ELF'))
print(platform.platform()) # Output: Detailed platform information
print(platform.uname()) # Output: Detailed system information
```

Note: you can import specific functions from the module without having to prefix the module name.

```
# Importing specific functions from the platform module
from platform import python_version, python_compiler,
python_implementation

# Using the imported functions without prefixing the module name
print(python_version()) # Output: Python version as a
string (e.g., '3.9.1')
print(python_compiler()) # Output: Python compiler used
print(python_implementation()) # Output: Python implementation
(e.g., 'CPython', 'PyPy')
```

2. **datetime Module:** The datetime module in Python provides functions for manipulating dates and times.

   a. **Current Date and Time:**

```
import datetime

now = datetime.datetime.now()

print(now)  # Output: Current date and time in YYYY-MM-DD
HH:MM:SS.ms format
```

   b. **Date Formatting:** The datetime object has a method for formatting date objects into readable strings.

```
import datetime

now = datetime.datetime.now()
formatted_date = now.strftime("%Y-%m-%d")
print(formatted_date)  # Output: Current date formatted as
YYYY-MM-DD
```

   c. **Date Arithmetic:**

```
from datetime import timedelta
import datetime

now = datetime.datetime.now()

print(now)
tomorrow = now + timedelta(days=1)
print(tomorrow)  # Output: Date and time for tomorrow
```

   d. **The strptime() Method:** The strptime() function in Python is used to parse a string into a datetime object based on a specified format. It takes two arguments:

1. The date/time string to be parsed.
2. A format string that defines the structure of the input string (e.g., "%Y-%m-%d %H:%M:%S").

   It returns a datetime object corresponding to the input string.

```
from datetime import datetime

date_string = "2023-06-01 08:15:30"

datetime_object = datetime.strptime(date_string, "%Y-%m-%d
%H:%M:%S")

print(datetime_object)
```

e. **The strftime() Method:** The datetime object has a method for formatting date objects into readable strings.The method is called strftime(), and takes one parameter, format, to specify the format of the returned string:

```
import datetime

x = datetime.datetime(2018, 6, 1)

print(x.strftime("%B"))
```

"%B" is a format code that instructs strftime to format the datetime object as the full month name (e.g., "June" for the month of June).

A reference of all the legal format codes:

Directive	**Description**	**Example**
%a	Weekday, short version	Wed
%A	Weekday, full version	Wednesday
%w	Weekday as a number 0-6, 0 is Sunday	3
%d	Day of month 01-31	31
%b	Month name, short version	Dec
%B	Month name, full version	December
%m	Month as a number 01-12	12
%y	Year, short version, without century	18
%Y	Year, full version	2018

Import specific functions from the module without having to prefix the module name.

```
# Importing specific function from the datetime module
from datetime import datetime

# Using the imported function without prefixing the module name
now = datetime.now()
print(now)  # Output: Current date and time
```

3. **math module:** The math module in Python provides a wide range of mathematical functions and constants. It includes functions for basic arithmetic, trigonometry, logarithms, combinatorics, and more.

The min() and max() functions can be used to find the lowest or highest value in an iterable:

```
x = min(5, 10, 25)
y = max(5, 10, 25)

print(x)  # Output: 5
print(y)  # Output: 25
```

The abs() function returns the absolute (positive) value of the specified number:

```
x = abs(-7.25)

print(x) # Output: 7.25
```

The pow(x, y) function returns the value of x to the power of y (x^y)

```
x = pow(4, 3)

print(x)  # Output: 64
```

a. **Basic Arithmetic Functions:**

1. math.sqrt(x): Returns the square root of x

```
import math

print(math.sqrt(16))  # Output: 4.0
```

2. math.ceil(x): Rounds a number upwards to its nearest integer

```
print(math.ceil(4.2))  # Output: 5
```

3. math.floor(x): Rounds a number downwards to its nearest integer

```
print(math.floor(4.8))  # Output: 4
```

4. math.fabs(x): Returns the absolute value of x.

```
print(math.fabs(-7.5))  # Output: 7.5
```

5. math.factorial(x): Returns the factorial of x.

```
print(math.factorial(5))  # Output: 120
```

b. Trigonometric Functions

1. math.pi: The mathematical constant π (pi), approximately 3.14159.

```
print(math.pi)  # Output: 3.141592653589793
```

2. math.sin(x): Returns the sine of x (x is in radians).

```
print(math.sin(math.pi / 2))  # Output: 1.0
```

3. math.cos(x): Returns the cosine of x (x is in radians).

```
print(math.cos(math.pi))  # Output: -1.0
```

4. math.tan(x): Returns the tangent of x (x is in radians).

```
print(math.tan(math.pi / 4))  # Output: 1.0
```

c. Logarithmic Functions

1. math.log(x[, base]): Returns the logarithm of x to the given base. If the base is not specified, returns the natural logarithm (base e) of x.

```
print(math.log(10))       # Output: 2.302585092994046 (natural
log)
print(math.log(100, 10))  # Output: 2.0 (log base 10)
```

2. math.log10(x): Returns the base-10 logarithm of x.

```
print(math.log10(1000))  # Output: 3.0
```

3. math.exp(x): Returns e raised to the power of x.

```
print(math.exp(2))  # Output: 7.38905609893065
```

d. Other Functions

1. math.pow(x, y): Returns x raised to the power of y

```
print(math.pow(2, 3))  # Output: 8.0
```

2. math.degrees(x): Converts angle x from radians to degrees.

```
print(math.degrees(math.pi))  # Output: 180.0
```

3. math.radians(x): Converts angle x from degrees to radians.

```
print(math.radians(180))  # Output: 3.141592653589793
```

4. math.e: The mathematical constant e, approximately 2.71828.

```
print(math.e)  # Output: 2.718281828459045
```

Import specific functions from the module without having to prefix the module name.

```
# Importing specific functions from the math module
from math import sqrt, ceil, floor, fabs, factorial

# Using the imported functions without prefixing the module name
print(sqrt(16))         # Output: 4.0
print(ceil(4.2))        # Output: 5
print(floor(4.8))       # Output: 4
print(fabs(-7.5))       # Output: 7.5
print(factorial(5))     # Output: 120
```

4. **random module:** The random module in Python provides functions to generate random numbers and perform random operations.

a. Basic Random Number Generation:

1. random.random(): Returns a random floating-point number between 0.0 and 1.0.

```
import random

print(random.random())  # Output: A random float between 0.0 and
1.0
```

2. random.randint(a, b): Returns a random integer between a and b (inclusive).

```
print(random.randint(1, 10))  # Output: A random integer between 1 and 10
```

3. random.uniform(a, b): Returns a random floating-point number between a and b.

```
print(random.uniform(1.5, 5.5))  # Output: A random float between 1.5 and 5.5
```

b. Random Choices from a Sequence

1. random.choice(seq): Returns a random element from the non-empty sequence seq.

```
fruits = ['apple', 'banana', 'cherry']
print(random.choice(fruits))  # Output: A random fruit from the list
```

2. random.choices(population, weights=None, k=1): Returns a list of k elements chosen from the population with optional weights.

```
print(random.choices(fruits, k=2))  # Output: A list of 2 random fruits
```

3. random.sample(population, k): Returns a list of k unique elements chosen from the population.

```
print(random.sample(fruits, 2))  # Output: A list of 2 unique random fruits
```

c. Random Shuffling: random.shuffle(x): Shuffles the sequence x in place

```
cards = ['ace', 'king', 'queen', 'jack']
random.shuffle(cards)
print(cards)  # Output: The shuffled list of cards
```

d. Setting the Seed: random.seed(a=None, version=2): Initializes the random number generator. If a is provided, it sets the seed, ensuring reproducibility.

```
random.seed(42)
print(random.random())  # Output: A reproducible random float
```

Import specific functions from the module without having to prefix the module name.

```
# Importing specific functions from the random module
from random import random, randint, choice

# Using the imported functions without prefixing the module name
print(random())             # Output: A random float between 0.0
and 1.0
print(randint(1, 10))       # Output: A random integer between 1
and 10 (inclusive)
print(choice(['apple', 'banana', 'cherry']))  # Output: A random
element from the list
```

5. **os module:** The os module in Python provides a way of using operating system-dependent functionality such as reading or writing to the file system, interacting with the environment, and managing processes.

1. os.getcwd(): Returns the current working directory.

```
import os

print(os.getcwd())  # Output: Current working directory
```

2. os.listdir(path='.'): Returns a list of entries in the directory given by path. The default is the current directory.

```
print(os.listdir())  # Output: List of files and directories in
the current directory
```

3. os.mkdir(path): Creates a directory named path.

```
os.mkdir('new_directory')
```

4. os.remove(path): Removes the file at path.

```
os.remove('file.txt')
```

5. os.rmdir(path): Removes the directory at path. The directory must be empty. If you attempt to remove a directory that is not empty, Python will raise an OSError with the message indicating that the directory is not empty.

```
os.rmdir('new_directory')
```

6. os.rename(src, dst): Renames the file or directory from src to dst.

```
os.rename('old_name.txt', 'new_name.txt')
```

7. os.name: Returns the name of the operating system.

```
print(os.name) # Output: It depends on the operating system
```

If the script is executed on a Linux system or macOS, the output would be:

```
posix
```

If the script is executed on a Windows system, the output would be:

```
nt
```

If os.name returns a value other than 'posix' or 'nt', it typically signifies an unsupported or unrecognized operating system environment.

Import specific functions from the module without having to prefix the module name.

```
# Importing specific function from the os module
from os import mkdir

# Using the imported function without prefixing the module name
mkdir('new_directory') # Creates a new directory named
'new_directory'
```

Understand what file handling is

File handling is an essential concept in programming that allows you to work with files on your computer's file system. It involves creating, reading, updating, and deleting files.

In Python, the built-in open() function is used to handle file operations.

Opening Files: To work with files in Python, you need to open them first.

1. The **open()** function is used to open a file, and it returns a file object.
   Syntax:

```
file_object = open(file_name, mode)
```

- file_name: Name of the file you want to open.
- mode: Mode in which you want to open the file. Common modes include:
- 'r': Read (default mode). Open the file for reading.
- 'w': Write. Opens a file for writing, creates the file if it does not exist
- 'a': Append. Opens a file for appending, creates the file if it does not exist
- "x" - Create. Creates the specified file, returns an error if the file exists

2. The **with open()** statement is used to properly close the file after its block of code is executed, even if the exception occurs. There is no need to close the file **file.close()** explicitly.
   Syntax:

```
with open('file_path', mode) as file:
     # Perform file operations
     pass
```

- file_path: Path of the file you want to open.
- mode: Mode in which you want to open the file.